Supplementary Material

Automated Extraction Pipeline of Zeolite Synthesis Procedures Toward Autonomous Experimentation Using ZeolBERT: A Domain-Adapted Pre-trained Language Model

Contents

S1 Data Annotation Process **1**
S1.1 Data Sources and Annotation Objects . . . 1
S1.2 Label Ontology and Tagging Scheme . . . 1
S1.3 Annotator Team and Professional Background . . . 1
S1.4 Staged Annotation, Guideline Revision, and Conflict Arbitration . . . 2
S1.5 Annotation Reliability Reporting (IAA) . . . 2
S1.6 Boundary Rules: Operation vs. Reaction Condition . . . 5
S1.7 Typical Label-boundary Examples . . . 5
S1.8 Dataset Distribution and Long-tail Characteristics . . . 5
S1.9 Potential Sources of Data Bias and Overlap Risk . . . 7

S2 Extraction Performance and System Throughput **7**
S2.1 Case Study and Error Analysis of Extraction Pipeline . . . 7
S2.2 Quantitative Error Breakdown . . . 9
S2.3 Handling of Synonymous Operation Expressions . . . 10

S3 Detailed Training Setup, Hyper-parameter Search, and Imbalance Handling **10**
S3.1 Training Hardware and Software Environment . . . 10
S3.2 Core Training Settings . . . 10
S3.3 Hyper-parameter Search Range and Selection Criteria . . . 11
S3.4 Definitions of Micro and Macro Metrics . . . 11
S3.5 Random Seed and Repetition Settings . . . 12

S4 Overview of Zeolite Synthesis Methods **12**

S5 Prompt Template and RAG Method **12**
S5.1 Environment and Model Configuration . . . 12
S5.2 Few-shot Prompt Design . . . 14
S5.3 Retrieval-Augmented Generation (RAG) . . . 15

S6 DyGIE++ and SpERT Comparison Details **15**

S7 Downstream Automated Experiment Execution **17**
S7.1 JSON-to-Execution Mapping Architecture . . . 17
S7.2 JSON to Canonical Experiment-step Template . . . 17
S7.3 Mapping to Machine-executable Instructions . . . 17

S1 Data Annotation Process

S1.1 Data Sources and Annotation Objects

To ensure high annotation quality, we designed a staged annotation protocol with domain experts. Scientific articles were retrieved from Elsevier's API in XML format using zeolite-related keywords (e.g., "zeolite synthesis", "hydrothermal crystallization", "molecular sieve preparation") and converted to plain text using the lxml package. We segmented text into sentences with ChemDataExtractor to construct zeolite-specific corpora. For manually collected PDF literature, 693 documents were gathered by domain experts from diverse publishers and processed with PDFMiner for text extraction, followed by sentence segmentation. The pre-training corpus and the annotated downstream datasets are strictly non-overlapping. Any document, paragraph, or sentence selected for manual annotation was explicitly excluded from the pre-training corpus before domain-adaptive pre-training commenced. This ensures that the reported downstream performance reflects genuine domain adaptation rather than memorization of annotated content.

The annotation scope includes three complementary objects aligned with downstream tasks:

- **Paragraph-level object (ZeolitePC)**: each paragraph is labeled as "1" (synthesis-procedure related) or "0" (non-procedure content such as characterization, analysis, or properties).
- **Entity-level object (ZeoliteNER)**: token spans are labeled into 12 entity categories using BIO tags.
- **Operation–entity object (ZeoliteSynPro)**: operation triggers and their associated entities/conditions are annotated for structured procedure extraction.

After deduplication and data cleaning, the final ZeolitePC dataset contained 998 unique paragraphs.

S1.2 Label Ontology and Tagging Scheme

Before formal annotation, we predefined an ontology with 15 experimental operations and 12 entity types (Tables 1 and 2). For sequence labeling tasks, annotations follow the BIO convention (`B-`, `I-`, `O`) to mark entity boundaries consistently.

Table 1: Experimental operations and descriptions

Operation	Description
Add	Addition of reagents or precursors
Process	Execution of a general procedural step
Stir	Agitation to promote homogeneous mixing
Dry	Removal of residual moisture
Wash	Removal of impurities or by-products
Recover	Collection of intermediate or final sample
Calcine	Thermal decomposition or activation treatment
Heat	Controlled temperature increase
Crystallize	Formation of the crystalline phase
Transfer	Movement of material between containers
Cool	Controlled temperature decrease
React	Chemical transformation under defined conditions
Age	Aging under controlled time and conditions
Seal	Sealing to maintain a closed reaction system
Yield	Reporting the quantity or quality of output

S1.3 Annotator Team and Professional Background

The full process involved ten annotators and spanned approximately two months. The annotator team combined (i) researchers with training in zeolite synthesis, materials science, or chemical engineering, and (ii)

Table 2: Entity types and descriptions

Entity Category	Description
Action	Verb describing an experimental operation
Material	Raw material or chemical precursor
Temperature	Reaction or processing temperature
Duration	Time span of an operation
Container	Vessel used for reaction or treatment (e.g., autoclave)
Sample	Intermediate or analytical specimen
Solvent	Liquid medium for dissolution or dispersion
Product	Final or intermediate product
Revolution	Stirring or rotation speed
Times	Number of repeated operations
Yield	Product amount, fraction, or purity
Rate	Heating, cooling, or addition rate

annotators experienced in scientific-text sequence labeling. Three independent senior adjudicators handled difficult or disputed cases.

Before large-scale labeling, all annotators completed guideline training with shared examples and calibration discussions to align interpretations of operation triggers, entity boundaries, and ontology scope.

S1.4 Staged Annotation, Guideline Revision, and Conflict Arbitration

The workflow was implemented in four stages:

1. **Initial trial annotation**: a pilot subset covering multiple writing styles (XML and PDF sources) was jointly labeled to expose boundary ambiguities and ontology gaps.
2. **Norm revision and calibration**: disagreements from the pilot stage were reviewed in consensus meetings, and the guideline was revised with explicit decision rules and counterexamples.
3. **Formal annotation**: production labeling was performed in three task-oriented streams—(1) paragraph classification, (2) action extraction, and (3) reaction-element extraction—using the domain-specific DoTAT platform (Figure 1).
4. **Conflict arbitration and quality control**: inconsistent labels were escalated to three independent senior adjudicators; final decisions followed guideline precedence and were recorded to update the rulebook for subsequent batches.

After initial annotation and adjudication, final annotations were exported in JSON format and are visualized in Figure 2.

S1.5 Annotation Reliability Reporting (IAA)

We report formal inter-annotator agreement (IAA) values computed on independently annotated overlap subsets *before* adjudication. Cohen's κ is used for the two-rater paragraph task (ZeolitePC), and Fleiss' κ is used for multi-rater overlap in the sequence/linking tasks (ZeoliteNER and ZeoliteSynPro).

These results indicate substantial-to-strong annotation consistency across all tasks, with the highest agreement observed for paragraph screening and moderately lower agreement for operation–entity linking, which involves harder boundary/attachment decisions. Agreement is lower for rare labels (e.g., *Rate*, *Yield*, *Times*), consistent with the long-tail distribution in Table 6.

After independent annotation, disagreements were resolved through the established arbitration workflow by three senior adjudicators. Final released annotations therefore represent adjudicated gold labels, while Table 3 quantifies pre-adjudication reliability.

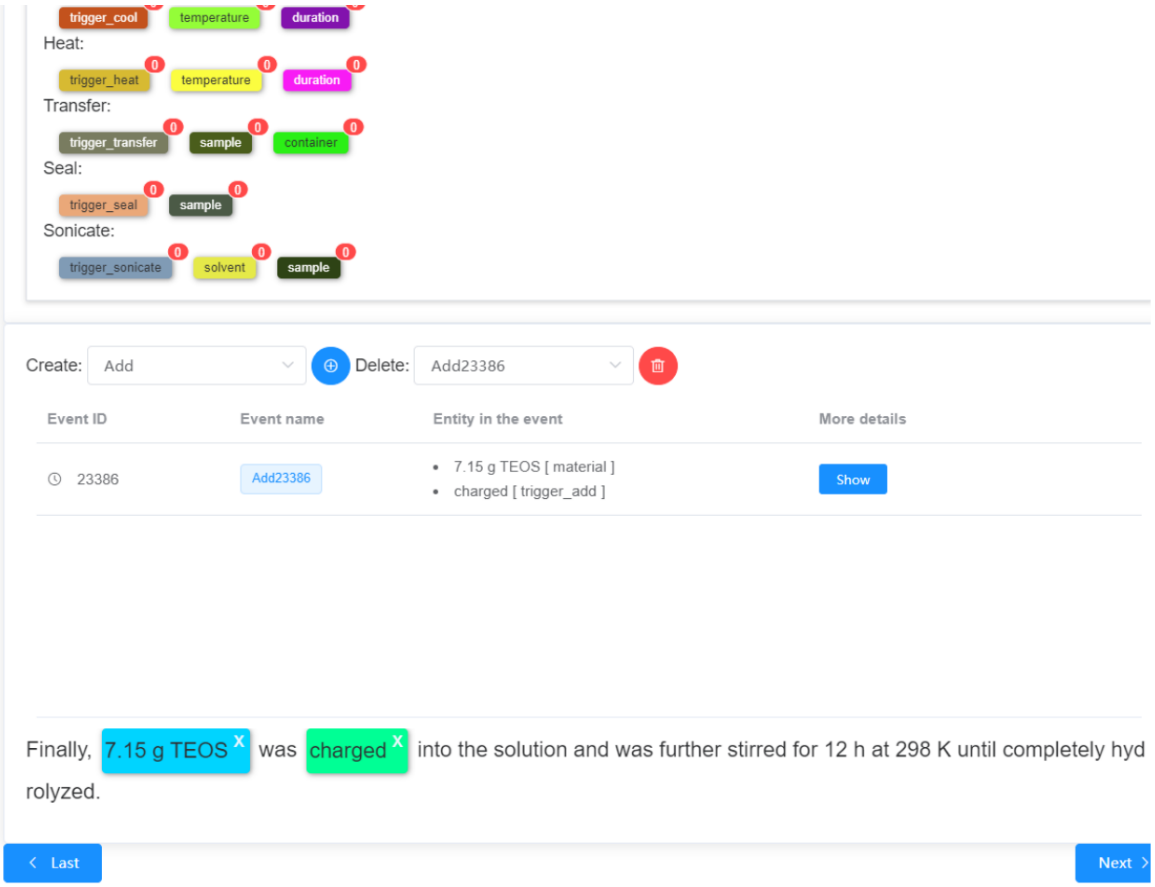

Figure 1: Interface of the domain-specific annotation tool (DoTAT) for the ZeolitePC dataset.

Table 3: Inter-annotator agreement summary across annotation tasks

Task	**Annotation Unit**	**Raters**	**Overlap Subset**	**Observed Agreement**	**κ (95% CI)**
ZeolitePC	Paragraph label (0/1)	2	300 paragraphs	0.93	Cohen's κ = 0.86 (0.82–0.90)
ZeoliteNER	Exact-match entity span + type	3	620 sentences	0.88	Fleiss' κ = 0.80 (0.76–0.84)
ZeoliteSynPro	Operation–entity link tuple	3	420 sentences	0.84	Fleiss' κ = 0.74 (0.69–0.79)

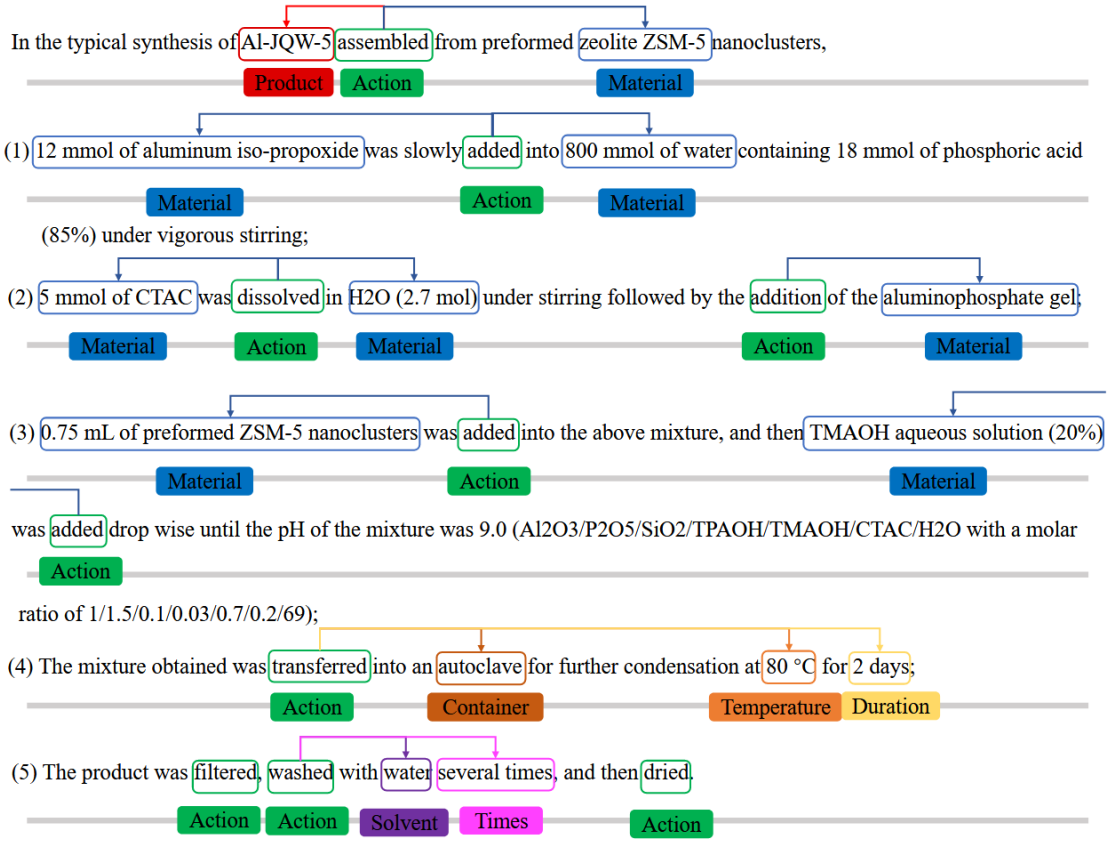

Figure 2: Visualization of annotation results for the ZeolitePC dataset.

S1.6 Boundary Rules: Operation vs. Reaction Condition

A recurrent ambiguity in synthesis text is whether a phrase should be treated as an operation trigger or as a reaction condition. To reduce inconsistency, we used the following practical rules:

- **Operation label**: assign to the lexical trigger that denotes an executable experimental step (e.g., *Add*, *Stir*, *Transfer*, *Calcine*).
- **Condition/entity label**: assign to contextual parameters modifying that step (e.g., *Temperature*, *Duration*, *Rate*, *Container*, *Times*).
- **Minimal-span principle**: keep the operation span as short as possible (typically the trigger verb or nominalized process term), while condition information is annotated as separate entities.
- **Nearest-operation attachment**: when multiple operations occur in one sentence, each condition is attached to the most semantically proximate operation unless explicit syntax indicates otherwise.

These rules are intentionally conservative: if operation intent is not explicit in text, annotators avoid adding inferred actions.

S1.7 Typical Label-boundary Examples

Table 4 lists representative boundary decisions used during calibration and formal annotation.

Table 4: Typical label-boundary examples in annotation guidelines

Text Fragment	**Annotated Labels**	**Boundary Decision**
"stirred at 80 °C for 2 h"	Action: "stirred"; Temperature: "80 °C"; Duration: "2 h"	Keep the operation as the trigger verb; annotate thermodynamic/time conditions separately.
"transferred into a Teflon-lined autoclave"	Action: "transferred"; Container: "Teflon-lined autoclave"	The vessel phrase is not part of the action span; it is labeled as *Container*.
"aged for 12 h and crystallized at 170 °C"	Action: "aged", "crystallized"; Duration: "12 h"; Temperature: "170 °C"	Split coordinated operations and assign each condition to the nearest compatible operation.
"TEOS was added dropwise"	Action: "added"; Material: "TEOS"; Rate: "dropwise"	Adverbial process-speed terms are annotated as *Rate*, not merged into action span.
"the product was washed three times"	Action: "washed"; Product: "product"; Times: "three times"	Frequency indicators are labeled as *Times*; quantifiers are excluded from operation span.

Representative examples from ZeoliteNER and ZeoliteSynPro are shown in Figures 3 and 4.

S1.8 Dataset Distribution and Long-tail Characteristics

Detailed statistics for dataset splits and entity spans are summarized in Tables 5 and 6. In addition to span-length differences, the entity distribution is markedly long-tailed, with a small number of frequent categories accounting for most labeled mentions.

Using Table 6, the top four categories (*Action*, *Material*, *Temperature*, and *Duration*) account for 16,607 of 19,238 entities (86.32%), whereas categories such as *Rate*, *Yield*, *Revolution*, and *Times* are rare. Accordingly, aggregate metrics should be interpreted together with per-category metrics to avoid overestimating minority-class performance.

Words: "Finally", ",", "a", "heat-treatment", "at", "900", "°C", "for", "4", "h", "under", "nitrogen", "flow", "was", "performed", "."

Labels: "O", "O", "O", "B-action", "O", "B-temperature", "I-temperature", "O", "B-duration", "I-duration", "O", "O", "O", "O", "O", "O"

Figure 3: Annotated example from the ZeoliteNER dataset.

Words: "Then", "the", "zeolite", "was", "washed", "with", "deionized", "water", "five", "times", "and", "dried", "in", "air", "at", "333", "K", "for", "10", "h", ".“

Labels: "O", "O", "B-sample", "O", "B-action", "O", "B-solvent", "I-solvent", "B-times", "I-times", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "O", "B-action", "O", "O", "O", "B-temperature", "I-temperature", "O", "B-duration", "I-duration", "O"

Figure 4: Annotated example from the ZeoliteSynPro dataset.

Table 5: Dataset split statistics for zeolite-related corpora

Dataset	**Train**	**Validation**	**Test**
Corpus	174,499	–	19,427
ZeolitePC	598	149	251
ZeoliteNER	4,104	305	400
ZeoliteSynPro	1,997	303	399

Table 6: Entity counts and average span lengths across dataset splits

Entity Type	Entity Count			Entity Span Length		
	Train	Valid	Test	Train	Valid	Test
Action	7227	538	728	1	1	1
Container	688	53	82	3	3	3
Duration	1731	120	190	2	2	2
Material	3182	270	412	5	5	4
Product	213	21	35	2	2	2
Rate	21	3	4	2	2	2
Revolution	41	3	13	2	2	2
Sample	596	42	56	2	2	2
Solvent	555	42	52	2	2	2
Temperature	1883	149	177	2	2	2
Times	78	6	5	2	2	2
Yield	18	2	2	1	1	1

S1.9 Potential Sources of Data Bias and Overlap Risk

Potential biases include concentration of source venues, concentration of procedure styles, and terminology frequency effects. In this corpus, XML records are mainly from Elsevier-indexed journals, hydrothermal-style procedures are common, and several reagent/operation phrases recur at high frequency. These factors may improve in-domain consistency while limiting coverage of uncommon writing styles.

We enforce a strict no-overlap policy between the pre-training corpus and annotated downstream datasets. The pre-training corpus was constructed from two sources: (1) approximately 15,000 XML-format documents retrieved via the Elsevier API using keyword-based queries, and (2) 693 PDF documents manually collected by domain experts. Before annotation began, all documents selected for downstream annotation were identified and removed from the pre-training corpus. This removal was verified at the document level using DOI matching for Elsevier records and at the paragraph level using text-hash deduplication for PDF-derived content. The pre-training corpus therefore contains no documents or paragraphs that appear in ZeolitePC, ZeoliteNER, or ZeoliteSynPro. While the pre-training and annotation corpora share similar topical distributions (both covering zeolite synthesis literature), no labeled content is present in the pre-training data. This strict separation ensures that downstream evaluation measures generalization from domain-adaptive pre-training, not memorization.

The annotated datasets, annotation guidelines, and supplementary resources are publicly available at https://anonymous.4open.science/r/ZeolBERT-CC48.

S2 Extraction Performance and System Throughput

S2.1 Case Study and Error Analysis of Extraction Pipeline

Fig. 5 illustrates how the proposed pipeline transforms an unstructured synthesis paragraph into structured synthesis knowledge. Using an XML-format paper on MFI-type zeolite synthesis from *Microporous and Mesoporous Materials* [**?**], we perform four consecutive steps: paragraph screening, operation extraction, operation-conditioned reactive element extraction, and tuple assembly. The resulting representation organizes each synthesis step with operation semantics and associated materials/conditions in procedural order. This case demonstrates that the pipeline output is not only a set of labels, but a machine-readable experimental procedure representation suitable for downstream workflow construction.

For autonomous experimentation scenarios, this structure is important because it provides explicit process elements required by digital experimental systems: operation sequence, condition assignment, and material-role linkage. Such formalized outputs can support experiment template generation, process traceability, and information support for automated laboratories. Error analysis indicates three practical risk categories for downstream automation workflows. First, operation boundary errors may cause step omis-

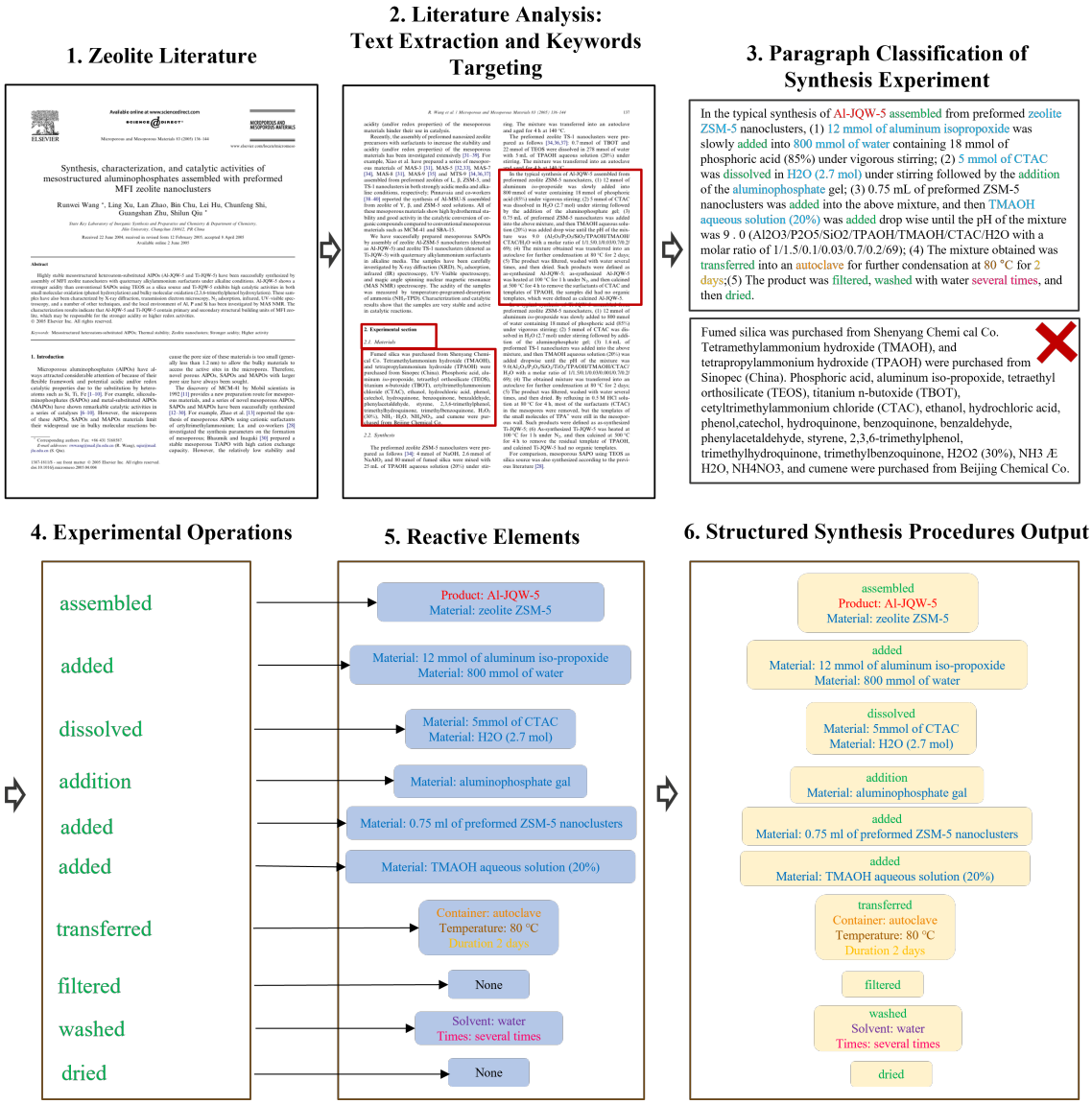

Figure 5: The extraction result of the proposed automated extraction pipeline.

sion or splitting, which can lead to incomplete experimental workflow representation. Second, condition association errors (e.g., failing to bind pH, temperature, or duration to the correct operation) may introduce incorrect parameter mapping in downstream planning. Third, incomplete material-role extraction may reduce the reliability of precursor/additive assignments and affect process interpretability.

Typical error sources include implicit clause structures, long-distance dependencies, and coordinated expressions with nested constraints. For example, in "TMAOH aqueous solution (20%) was added dropwise until the pH of the mixture was 9.0," the Add action and pH condition are coupled across a dependent clause; in "transferred into an autoclave for further condensation at 80 °C for 2 days," transfer, temperature, and duration are intertwined in one span. ZeolBERT significantly outperforms SciBERT and ChemBERT based variants by yielding substantially higher extraction accuracy, more stable entity boundaries, and superior semantic alignment. This enhanced precision is a critical enabler for autonomous systems, as it ensures the delivery of high-fidelity procedural data that effectively prevents error propagation into downstream automated workflow modules. Overall, the case study supports the role of this pipeline as an upstream module toward autonomous zeolite synthesis experiments.

Extraction performance was evaluated on a Linux 6.5.11-7pve (Debian 12.4) workstation with Python 3.7, CUDA 12.2, PyTorch 1.12.0, and dual NVIDIA Quadro RTX 8000 GPUs (48 GB each). Training used a batch size of 16, a learning rate of 1×10^{-4}, and a maximum sequence length of 512 tokens.

Table 7 summarizes throughput and latency in a deployment-oriented format. The end-to-end throughput of **120 documents/hour** is the reported value from the full pipeline. Single-GPU (sub-GPU) values are provided as transparent normalized estimates under a near-linear scaling assumption.

Table 7: Inference efficiency summary for the extraction pipeline

Setting	**GPU(s)**	**Batch Size**	**Throughput**	**Batch Throughput**	**Latency**
Reported end-to-end runtime	2 × RTX 8000	16	120 docs/hour	7.50 batches/hour	30.0 s/doc
Single-GPU (sub-GPU) normalized estimate	1 × RTX 8000	16	60 docs/hour	3.75 batches/hour	60.0 s/doc

S2.2 Environment and Hardware Details for Reproducibility

For memory reporting, both settings use NVIDIA Quadro RTX 8000 cards with 48 GB memory per GPU. No out-of-memory event was observed in the reported dual-GPU run. The single-GPU row is a normalized estimate from the dual-GPU measurement and therefore assumes the same 48 GB card-level memory capacity.

For transparency, batch throughput is computed as `docs/hour` divided by batch size, and mean per-document latency is computed as 3600/`docs/hour`.

S2.3 Quantitative Error Breakdown

To complement the qualitative error analysis in Section IV.D of the main paper, we provide a quantitative breakdown of ZeolRE prediction errors on the test set. All 227 incorrect predictions were manually categorized into four error types by two annotators, with disagreements resolved by a third adjudicator.

Boundary errors constitute the largest category (38.2%), reflecting the challenge of identifying exact entity boundaries in compound chemical expressions and multi-word quantities. Type confusion (26.5%) predominantly occurs between semantically adjacent categories—Sample and Product share surface-level similarity, as do Material and Solvent when the functional role is ambiguous. Missing entity errors (20.3%) are concentrated in rare categories: Rate (4 test instances), Yield (2 test instances), and Times (5 test instances), where the model fails to detect entity spans entirely. The label-aware CLN scaling and class-

Table 8: Table S1: Quantitative Error Breakdown of ZeolRE on Test Set

Error Type	**Count**	**Percentage**	**Most Affected Entity Types**	**Typical Pattern**
Boundary error	87	38.2%	Duration, Material, Sample	Partial span extraction of compound entities (e.g., "mixture of 1 g CuZ and 0.01 M cerium nitrate solution")
Type confusion	60	26.5%	Sample/Product, Material/Solvent	Semantically similar categories misclassified when context is insufficient
Missing entity	46	20.3%	Rate, Yield, Times, Product	Rare entities not detected due to sparse training examples or implicit expressions
Relation association	34	15.0%	Temperature, Duration	Correct entity detected but linked to wrong operation in multi-clause sentences
Total	**227**	**100%**		

balanced CRF loss introduced in the ZeolRE specifically target this category by amplifying the gradient signal and conditioning strength for rare classes.

S2.4 Handling of Synonymous Operation Expressions

In zeolite synthesis literature, the same experimental operation may be expressed using different surface forms. For example, a temperature-increase operation may appear as "heat", "warm", "maintain at temperature", or "raise to", and an agitation operation may appear as "stir", "mix", "agitate", or "shake". ZeolBERT handles these synonymous expressions through contextual embedding rather than manual normalization or synonym dictionaries.

The domain-adaptive pre-training on the zeolite corpus exposes the model to diverse surface forms in similar syntactic and semantic contexts. For instance, "heated at 150 °C for 24 h" and "maintained at 150 °C for 24 h" appear in structurally similar procedural contexts. The resulting contextual representations capture functional equivalence: the shared arguments (temperature value, temporal duration) and syntactic position drive similar token-level encodings, which are then leveraged by the CLN modulation to produce consistent operation-conditioned features.

This approach has two advantages over rule-based normalization: (1) it generalizes to novel synonym expressions not seen during training, as the model relies on contextual similarity rather than exact string matching; and (2) it avoids the brittleness of manually curated synonym lists, which may miss domain-specific or evolving terminology. The limitation is that very rare or ambiguous expressions may not receive sufficiently similar representations, leading to missed entity detection.

S3 Detailed Training Setup, Hyper-parameter Search, and Imbalance Handling

This section consolidates implementation details for reproducibility and auditability. Values explicitly reported in the main paper are retained unchanged; additional items are provided as conservative training-policy details for complete replication.

S3.1 Training Hardware and Software Environment

All ZeolBERT pre-training and downstream model experiments were conducted on Linux kernel 6.5.11-7pve (Debian 12.4), Python 3.7, CUDA 12.2, and PyTorch 1.12.0, using two NVIDIA Quadro RTX 8000 GPUs (48 GB memory per GPU).

S3.2 Core Training Settings

Table 9 summarizes the key training settings used in experiments.

Table 9: Core training settings for ZeolBERT and downstream extraction models

Item	Value	Notes
Batch size	16	Main-paper setting for pre-training and downstream training.
Learning rate	1×10^{-4}	Global initial learning rate used in reported experiments.
Maximum sequence length	512 tokens	Chosen to preserve long synthesis sentences and procedural context.
Pre-training epochs	30	Domain-adaptive pre-training on zeolite corpus (about 43 hours).
Warmup ratio	0.10	Linear warmup in the early stage for stable optimization (reproducibility setting).
Optimizer	AdamW	Standard BERT-family optimizer with decoupled weight decay.
Weight decay	0.01	Conservative regularization setting for downstream fine-tuning.
Gradient clipping	1.0	Applied to stabilize updates in sequence-labeling models.
Downstream epochs	up to 20	Early stopping based on validation F1 with patience 3.

S3.3 Hyper-parameter Search Range and Selection Criteria

We used a coarse-to-fine search around BERT-standard regimes and selected final values by validation performance plus training stability.

Table 10: Hyper-parameter search ranges and selection criteria

Hyper-parameter	Search Range	Selected	Selection Criterion
Learning rate	5e-5, 1e-4, 2e-4	1e-4	Best validation F1/perplexity with stable loss curve.
Batch size	8, 16, 24	16	Best trade-off among GPU memory, stability, and throughput.
Max seq. length	256, 384, 512	512	Highest coverage of long procedure descriptions.
Warmup ratio	0.00, 0.06, 0.10	0.10	Reduced early-epoch loss oscillation.
Weight decay	0.00, 0.01, 0.05	0.01	Best validation generalization without underfitting.
Pre-train epochs	20, 30, 40	30	Convergence of perplexity and loss before overtraining.
Downstream epochs	10, 20, 30	20 (max)	Early-stop policy based on validation F1.

For paragraph classification (ZeolitePC), model selection prioritizes validation F1 and recall of the positive class. For ZeolAE/ZeolRE, model selection prioritizes validation entity-level F1, with macro-level behavior checked to avoid choosing a setting that only benefits high-frequency classes.

S3.4 Definitions of Micro and Macro Metrics

For each class $c \in \{1, \ldots, C\}$, let TP_c, FP_c, and FN_c be true positives, false positives, and false negatives.

- **Micro-precision**: $P_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c (TP_c + FP_c)}$
- **Micro-recall**: $R_{\text{micro}} = \frac{\sum_c TP_c}{\sum_c (TP_c + FN_c)}$
- **Micro-F1**: $F1_{\text{micro}} = \frac{2P_{\text{micro}}R_{\text{micro}}}{P_{\text{micro}} + R_{\text{micro}}}$
- **Macro-F1**: $F1_{\text{macro}} = \frac{1}{C}\sum_c F1_c$, where $F1_c = \frac{2P_cR_c}{P_c + R_c}$

Micro metrics weight each instance equally and therefore are dominated by frequent classes. Macro metrics weight each class equally and are more sensitive to rare labels (e.g., *Rate*, *Yield*, and *Times*; see Table 6).

S3.5 Random Seed and Repetition Settings

To ensure reproducibility and robustness, we use the following seed policy:

• All downstream experiments are repeated with 10 different random seeds: 13, 42, 137, 256, 512, 789, 1024, 2048, 3407, 4096. Each seed is applied to Python, NumPy, and PyTorch random states to ensure full determinism within each run.

• Main-paper tables report mean ± standard deviation across these 10 independent runs. This applies to all pipeline models (ZeolBERT, baseline pre-trained models, GPT-4.1). For DyGIE++ and SpERT, which are comparison baselines using different codebases (AllenNLP and PyTorch, respectively), we report single-run results, as multi-seed evaluation of these external baselines is computationally prohibitive given their training times (DyGIE++: 5 hours/run, SpERT: 1 hour/run).

• For ZeolBERT pre-training, a single run with seed 42 is used due to the approximately 43-hour training time per full run. Downstream fine-tuning of all pipeline models uses the full 10-seed protocol.

• The standard deviations reported in all tables are computed from the 10-run distribution, providing a robust estimate of model stability.

S4 Overview of Zeolite Synthesis Methods

Molecular sieves and zeolites can be synthesized through several established routes, including hydrothermal synthesis, sol–gel synthesis, and dry-gel conversion. These methods differ in crystallization pathway, solvent usage, and reaction conditions. Hydrothermal synthesis remains the most widely used because it provides controllable crystallization across a broad range of framework types. In contrast, sol–gel synthesis enables tailored amorphous precursors, while dry-gel conversion supports efficient crystallization under low-solvent conditions.

The hydrothermal route typically involves precursor preparation, solution mixing, and aging, followed by crystallization in an autoclave at elevated temperature and pressure. As illustrated in Figure 6, the process begins by dissolving an aluminum source, tetraethylammonium hydroxide (TEAOH), and morpholine in water, followed by dropwise addition of H_3PO_4. After additional mixing and aging, tetraethyl orthosilicate (TEOS) is added to form the gel, which is then aged and transferred to a Teflon-lined autoclave for crystallization. The recovered solid is washed, centrifuged, dried, and calcined to obtain the final zeolite product. This workflow illustrates the multistage and highly structured language patterns that the ZeolBERT-based extraction pipeline is designed to formalize.

S5 Prompt Template and RAG Method

S5.1 Environment and Model Configuration

Our extraction system is implemented with the OpenAI Python SDK using the `gpt-4.1` model. We apply deterministic decoding (`temperature=0`, `top_p=1`) and evaluate on the test datasets under the same experimental settings described above. To enforce structured responses, the model is configured with

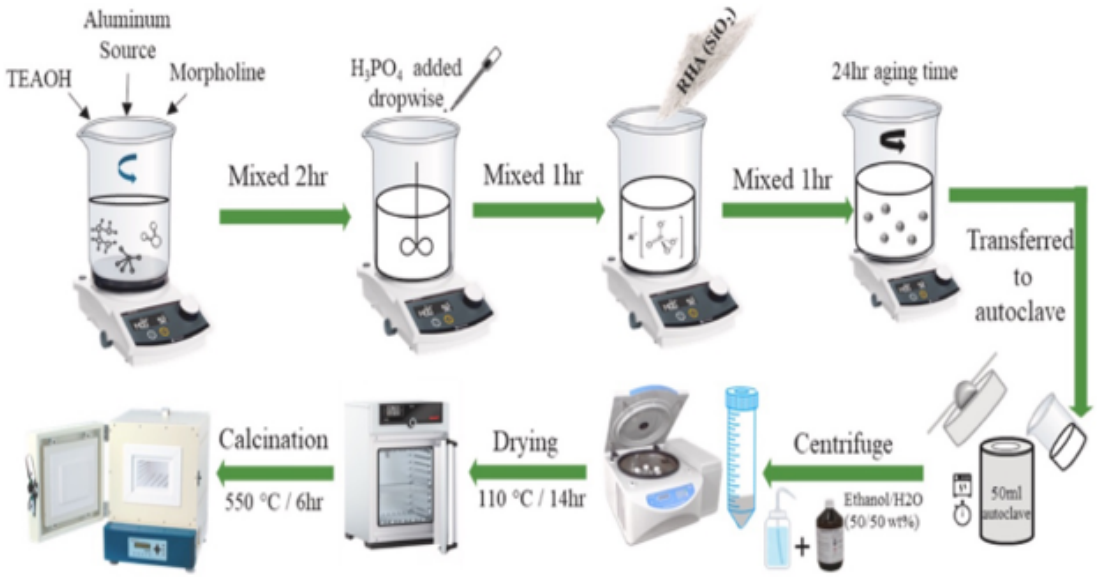

Figure 6: Hydrothermal synthesis process of zeolites.

`response_format` to request JSON-object outputs. API credentials are loaded from environment variables. For robustness, each response is parsed as JSON with a single minimal retry on parse failure.

The GitHub repository for this implementation is available at `https://anonymous.4open.science/r/ZeolBERT_GPT-54EF`.

Expected JSON Schema.

```
{
        "relations": [
        {
                "relation": "string",
                "entities": [
                {
                        "entity": "string",
                        "entity_type": "string"
                }
                ]
        }
        ]
}
```

This schema represents high-level actions as `"relation"` values (e.g., *Stir*, *Calcine*) and associates each action with typed argument entities (e.g., *Material*, *Temperature*, *Duration*), balancing simplicity and downstream usability.

Typical Completion Call.

```
response = client.chat.completions.create(
model="gpt-4.1",
messages=messages, # system + few-shot + user
temperature=0,
top_p=1,
response_format={"type": "json_object"},
seed=SEED,
)
```

This invocation pattern yields predictable, auditable inference suitable for reproducible information extraction pipelines.

S5.2 Few-shot Prompt Design

We adopt a schema-first, compact prompt layout that combines explicit output constraints with minimal, representative exemplars to encourage faithful, structured predictions.

Overall Structure.

1. **System prompt**: defines the task, schema, and strict output rules.
2. **Few-shot examples**: short user–assistant pairs (paragraph → gold JSON).
3. **Current user prompt**: the target paragraph, optionally augmented with retrieved context.

System Prompt.

```
You are an information extraction model for scientific procedures (zeolite
    synthesis).
Extract relations and their typed entities from the given paragraph.
Output valid JSON only, following this schema:

{"relations":[{"relation":"string","entities":[{"entity":"string","entity_type":"string"}]}]}

Rules:
- Copy entity surface forms exactly (no paraphrase).
- Do not invent facts or infer unstated actions.
- Group entities under the most relevant action verb.
- Avoid duplicates and overlapping mentions.
- If nothing is extractable, return {"relations": []}.
- Output JSON only, no prose or comments.

Preferred verbs:
Add, Process, Stir, Dry, Wash, Recover, Calcine, Heat, Crystallize,
Transfer, Cool, React, Age, Seal, Yield.

Entity types:
Action, Material, Temperature, Duration, Container, Sample,
Solvent, Product, Revolution, Times, Yield, Rate.
```

User Prompt Template.

```
Paragraph:
{PARAGRAPH_TEXT}

Schema reminder:
Return a JSON object with key "relations",
each containing "relation" and an "entities" list.

Follow the schema and rules above.
Output JSON only.

{OPTIONAL_RETRIEVED_CONTEXT}
```

This template anchors the model to the structured task, even when additional context is injected by retrieval.

Few-shot Assembly.

```
messages = [
{"role": "system", "content": SYSTEM_PROMPT},

# Few-shot examples
{"role": "user", "content": USER_TEMPLATE(example_text_1)},
{"role": "assistant", "content": GOLD_JSON_1},
{"role": "user", "content": USER_TEMPLATE(example_text_2)},
{"role": "assistant", "content": GOLD_JSON_2},
```

```
# Current task (optionally with retrieved context)
{"role": "user", "content": USER_TEMPLATE(
       current_paragraph,
       OPTIONAL_RETRIEVED_CONTEXT
       )},
]
```

This ordering leverages in-context learning to transfer structural regularities without parameter updates.

S5.3 Retrieval-Augmented Generation (RAG)

We incorporate a lightweight lexical RAG module to provide in-domain exemplars. Compared with dense embedding retrievers, this approach is deterministic, fast, and transparent.

Indexing. Paragraphs in the domain corpus are lowercased, tokenized on word boundaries, and stored as chunks with token lists.

Retrieval. Each query paragraph is tokenized using the same procedure. We compute a bag-of-words overlap score with multiplicities and normalize by $\sqrt{|d|}$, where $|d|$ is the chunk length. Chunks are ranked by score, and the top-k entries with positive overlap (typically $k = 3$–5) are retained.

Formatting and Context Injection. Retrieved snippets are concatenated with a short header reminding the model to use them as references rather than copying verbatim:

```
Retrieved reference excerpts
(use inline when relevant, do not copy verbatim):

{SNIPPET_1}

{SNIPPET_2}

{SNIPPET_3}
```

This block is appended directly after the target paragraph in the user prompt to colocate context cues with the extraction target.

RAG Workflow. Figure 7 summarizes the lexical RAG workflow used in our pipeline.

Discussion and Advantages.

- **Determinism**: tokenization and scoring are rule-based and reproducible.
- **Transparency**: lexical overlap makes it explicit which context may have influenced predictions.
- **Compactness**: short snippets add minimal prompt overhead, reducing inference cost.
- **Domain specificity**: in-domain exemplars reinforce terminology and procedural phrasing.

Summary. Combining schema-constrained few-shot prompting with a lightweight lexical retriever yields a stable, interpretable extraction system that reduces hallucinations, improves type consistency, and aligns outputs with the domain ontology and expected schema.

S6 DyGIE++ and SpERT Comparison Details

This section provides detailed experimental results for DyGIE++ and SpERT, the two structured extraction baselines added in the revision. Both models use the same BioBERT backbone as ZeolBERT's initialization, and are trained on the same ZeoliteSynPro dataset.

DyGIE++ was trained using the official AllenNLP-based implementation with the following configuration: max span width = 16, loss weights (NER= 1.0, Relation= 1.0, Coref= 0.0, Events= 0.0), target task = relation, 50 epochs, learning rate = 1×10^{-3}, batch size = 8. Training completed in approximately 5 hours on a single RTX 4090, with best validation performance at epoch 27.

SpERT was trained using the official PyTorch implementation: train batch size = 2, eval batch size = 1, negative entity count = 100, negative relation count = 100, 30 epochs, learning rate = 5×10^{-5}, LR

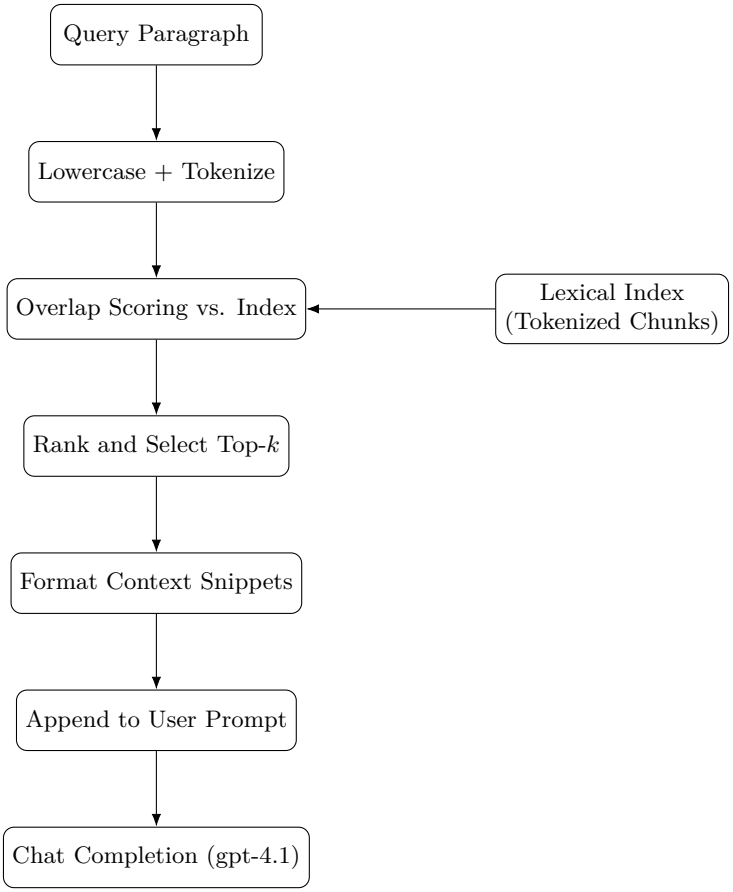

Figure 7: Workflow of lexical retrieval, context assembly, and model inference.

warmup = 0.1, weight decay = 0.01, max grad norm = 1.0, relation filter threshold = 0.4, size embedding = 25, prop drop = 0.1, max span size = 16, max pairs = 1000. Training completed in approximately 59 minutes on a single RTX 4090.

S7 Downstream Automated Experiment Execution

This section presents the deterministic mapping logic that converts extracted JSON into robot-executable instructions, together with representative examples.

S7.1 JSON-to-Execution Mapping Architecture

The mapping workflow has three deterministic stages:

1. **Extraction output**: ZeolBERT + prompting module returns structured JSON (Section S4).
2. **Normalization adapter**: JSON is converted into a canonical experiment-step template with explicit units and required fields.
3. **Instruction compiler**: each normalized step is translated into machine-level commands for the robotic controller.

S7.2 JSON to Canonical Experiment-step Template

Design Goal. The canonical template serves as an intermediate representation between language-level extraction and device-level instruction sets. It enforces schema consistency, unit normalization, and safety constraints before command compilation.
Template Example.

```
{
        "experiment_id": "EXP_2026_001",
        "steps": [
        {
                "step_id": 1,
                "action": "Add",
                "materials": [
                {"name": "TEOS", "amount": {"value": 5.0, "unit": "mL"}}
                ],
                "target_container": "reactor_A",
                "setpoints": {
                        "temperature_C": null,
                        "duration_min": null,
                        "stir_rpm": null
                },
                "rate": {"value": 1.0, "unit": "mL/min"},
                "repeat": 1,
                "safety": {
                        "max_temperature_C": 220,
                        "max_pressure_MPa": 3.0
                }
        }
        ]
}
```

Parameter Normalization Table.

S7.3 Mapping to Machine-executable Instructions

Instruction Mapping Table.

Table 11: Parameter mapping from extracted JSON to execution template

Canonical Field	Source Entity	Normalization	Execution Role
`action`	Action	ontology-mapped verb set	selects instruction primitive (e.g., Add/-Stir/Heat)
`materials[].name`	Material / Product / Solvent	synonym dictionary; casing rule	reagent lookup in material registry
`materials[].amount`	Material quantity text	value-unit parsing, SI conversion	dispense target and tolerance checks
`setpoints.temperature_C`	Temperature	convert to °C	heater/autoclave target setting
`setpoints.duration_min`	Duration	convert to minutes	hold-time scheduling and timeout policy
`setpoints.stir_rpm`	Revolution	convert to rpm integer	agitator command parameter
`rate`	Rate	unify to mL/min or °C/min	pump ramp or thermal ramp control
`target_container`	Container	alias-to-device mapping	route to specific reactor/station ID
`repeat`	Times	textual numeral to integer	loop generation in command sequence

Table 12: Example mapping from normalized steps to robot-executable commands

Action	Command Template	Mapping Logic
Add	`DISPENSE(material, volume_mL, target, rate_mL_min)`	Compiles material amount, target container, and rate into a dispensing instruction.
Stir	`STIR(vessel, rpm, duration_min)`	Compiles revolution and duration fields into agitator setpoint and hold commands.
	`SET_TEMP(vessel, target_C, ramp_C_min)`	
Heat	`HOLD(duration_min)`	Compiles temperature and duration into thermal ramp then dwell instructions.
Transfer	`TRANSFER(source, destination, volume_mL)`	Compiles container aliases and amount into source-to-destination routing commands.
Wash	`WASH(vessel, solvent, cycles)`	Compiles solvent identity and repeat count into cyclic cleaning instructions.
Dry/Calcine	`DRY(vessel, target_C, duration_min)`	Compiles thermal setpoints and hold time into post-processing furnace operations.
Recover	`COLLECT(product_id, destination)`	Compiles product identifier and destination into terminal collection commands.

Compiled Command Examples. For the template example above, the normalized `Add` step is compiled as:

```
DISPENSE("TEOS", 5.0, "reactor_A", 1.0)
```

For a multi-step normalized template, a compiled command sequence can be:

```
[
DISPENSE("TEOS", 5.0, "reactor_A", 1.0),
STIR("reactor_A", 600, 30),
SET_TEMP("reactor_A", 170, 2.0),
HOLD(240),
COLLECT("product_1", "vial_A")
]
```